



脸在电脑里，一步都不出去

Faceplugin「Open Source Face Recognition SDK」小白指南 —— 本地离线人脸识别全链路拆解 + 真机验货

detections→landmark→align→embed→compare 五步流水线

纯 CPU 单脸约 50ms

全部权重仅 61MB

⚠️ "开源"有水分：核心对齐是闭源 DLL，且无 LICENSE

一句话总结：这是一个把**人脸检测** → **68点关键点** → **人脸对齐** → **特征提取** → **相似度比对**整条流水线打包成 Python 函数的 SDK。宣传卖点"100% On-Premise（数据不出设备）"**实测属实**——不用网、不用显卡，CPU 上单张脸全流程约 50 毫秒。但"开源"成色要打折：检测/关键点/特征三段是真 PyTorch 源码，**人脸对齐+姿态估计藏在 70KB 的闭源 DLL 里，仓库连 LICENSE 文件都没有**。学它最大的价值：一次看懂人脸识别的完整骨架，顺便学会给"开源项目"验货。

Q1

人脸识别分几步？

Q2

"开源"成色如何验货？

Q3

1MB 检测器怎么跑？

Q4

68 个关键点怎么来？

Q5

为什么必须对齐？

Q6

IR-50 与 256 维特征

Q7

相似度公式的秘密

Q8

闭源姿态估计做啥？

Q9

CPU 性能实测账单

Q10

验货实验复盘

Q11

数据不出设备的边界

Q12

能拿它做什么？

Q1 一次"人脸识别"到底分几步?

很多人以为人脸识别是"一个 AI 看一眼就认出你"。实际上它是**五道工序的流水线**，每道工序各管一件事，这个 SDK 的代码目录结构就是按这五步摆的：

工序	代码目录	输入→输出	类比
①检测 Detect	face_detect/	整张图 → 人脸框(x,y,w,h)+置信度	人海里先圈出"哪里有脸"
②关键点 Landmark	face_landmark/	人脸区域 → 68 个坐标点	标出眼角、鼻尖、嘴角
③对齐+姿态 Align/Pose	face_util/ (闭源DLL)	原图+关键点 → 摆正的 128×128 小图	把歪头照片"扶正"
④特征 Embed	face_feature/	摆正小图 → 256 个数字的向量	把脸变成"身份证号"
⑤比对 Compare	run.py	两个向量 → 0~100 相似分	两张身份证号对一下



为什么要拆成流水线？ 因为每一步都要为下一步"减负"：检测把整图缩小成脸的小框，关键点把框变成几何坐标，对齐把任意角度的头扶正，最后特征提取只需面对"标准姿势的标准小图"。分工越细，每个模型越小越快——这正是它能 CPU 实时跑的原因。

Q2 自称"Open Source", 成色到底如何?

推文说它"开源、免费、离线"。逐项验货结果如下——**三真一假一缺失**：

宣传	实货	判定
检测/关键点/特征	真 PyTorch 源码 + 权重全在仓库里，可直接读、可改造	✔ 真
100% On-Premise 离线	实测断网可跑，无任何网络请求	✔ 真
免费、CPU 可跑	实测纯 CPU 单脸 50ms，无 key 无计费	✔ 真
"开源 SDK"	人脸对齐 align_vertical 与姿态 getPose68 编译在 FaceUtil.dll / libFaceUtil.so (70KB 闭源二进制) 里，只有 ctypes 调用壳	✘ 灰色
License-Open Source 徽章	仓库里 根本没有 LICENSE 文件 ——法律上默认"保留所有权利"，商用有风险	⚠ 缺失

另一个宣传缺口：README 和官网宣传了**活体检测、防照片攻击、表情/属性分析**，但仓库代码里只有"检测+关键点+特征+比对"四样。README 的 "Current Capabilities" 自己也只列了这四项。宣传页 ≠ 仓库实货，看项目要以**代码**为准。

怎么给"开源"验货？三查：① 查 **LICENSE** 文件在不在、是哪种协议 (MIT/Apache 随使用，AGPL 有传染性)；② 查关键功能是源码还是 **.dll/.so/.exe** 二进制；③ 查模型权重是否随仓库发布 (要外链下载的，外链挂了项目就死)。本项目三条里中了两枪。

Q3 1MB 的人脸检测器，凭什么能在 CPU 上跑？

它用的检测模型叫 **MB-Tiny SSD (slim-320)**，权重只有 **1.09MB**——比一张手机照片还小。这份代码和权重命名与 Linzaer 的《Ultra-Light-Fast-Generic-Face-Detector-1MB》开源项目完全同源（在原项目里叫 version-slim-320.pth）。

它快的秘诀有两个：

- ① **骨干网极小**：用深度可分离卷积（每个通道单独卷、不再跨通道混合，计算量降一个数量级）堆出的 MobileNet 式骨架，全网络只有几十万参数（普通 ResNet-50 有 2500 万）；
- ② **SSD 单阶段+多尺度**：不做“先出候选框再精修”两步，而是卷积过程中在 5 个不同大小的特征网格上一次性预测，每个网格格子里放 2 个预设框（anchor），网络只需微调位置和输出“是不是脸”。



实测：一张 860×520 的图检测一张脸只要 **约 20ms (CPU)**，检测置信度直接给到 1.000。代价是精度上限不如 RetinaFace 等大模型，且输入被压到 320×320，特别小的脸会漏——个人项目够用，安防级需求要换更强的检测器。

Q4 68 个关键点是怎么“回归”出来的？

关键点模型是 **MobileFaceNet** 结构，输入是从人脸框裁出的 **64×64 灰度小图**，输出层直接吐出 **136 个数字 = 68 个 (x,y) 坐标**。注意：它不是“找到”这些点，而是**一步算出（回归）**这些坐标数值——分类网络输出“哪个类别”，它输出 136 个连续数值，结构上是同一套卷积网络，只是最后一层的神经元个数改成了 136。



源码里的巧思：模型输出的是 0~1 的**相对坐标**（占裁剪图的比例），代码最后一步 `landmark[:,0]*iExFaceW + iExFaceX` 把它乘回裁剪区宽、加回裁剪区左上角，换算成原图绝对坐标——“神经网络算相对值，确定性代码管换算”，两层各干各的。实测每张脸 68 点只要 **6ms**。

Q5 为什么必须先"对齐"? 不摆正直接提特征行不行?

同一张脸, 仰头 30° 拍和正面拍, 像素差异大得像两个人。如果直接把整张脸喂给特征网络, 网络学到的大量信息是"头的角度"而不是"这个人是谁"。对齐 (Alignment) 就是 **利用关键点把任意角度的脸"焊接"回标准姿势**——眼睛永远在同一高度, 鼻尖永远居中。

不对齐: 角度当身份

歪 30° 的脸 → 向量 A

正脸 → 向量 B

A、B 离得远 X

VS

对齐后: 只剩"人"的差异

歪脸→扶正→标准小图→向量 A'

正脸→扶正→标准小图→向量 B'

A'、B' 靠得近 ✓ (同一人)

这个 SDK 的对齐函数 `align_vertical` 接收原图 + 关键点, 按"两眼水平、脸部垂直"的标准仿射变换输出一张 **128×128 摆正小图**, 再送进特征网络。它和姿态估计一起, **编译在那个闭源 DLL 里** (见 Q2/Q8)。

落回术语: 对齐在学术上叫"人脸规范化 (face normalization)", 本质是一次**仿射变换** (旋转+缩放+平移, 直线变换后仍是直线)。变换参数从哪来? 就从 Q4 的关键点来——这就是为什么 ② 和 ③ 在流水线上必须紧挨着。

Q6 IR-50 与 256 维特征: 一张脸如何变成"身份证号"?

特征模型 **IR-50** = 带 **IR (Improved Residual)** 模块的 **ResNet-50**——InsightFace 项目的人脸骨干网标准配方: 把残差块里的卷积换成"先降维 1×1、3×3、再升维 1×1"的瓶颈结构并去掉偏置。它的训练目标很特别: **不是"认出这是谁", 而是"把同一个人拉近、把不同的人推远"** (度量学习, 如 ArcFace 损失)。所以训练完把最后的分类层扔掉, 倒数第二层输出就是人人可用的"脸向量"。

两个实测发现: ① 输出是 **256 维** (多数主流方案是 512 维, 这是它自己裁过的版本); ② 向量做了 **L2 归一化** (实测范数=1.0000), 即只留方向、抹掉长度——比较两张脸变成纯比较方向。

还有一个"古董级"细节: 它的权重不是常规的 `.pth`, 而是一个 56MB 的 `irn50_pytorch.npy`——一个 numpy 字典存着每个卷积层的名字和数值, 代码在启动时用 `__conv()` / `__batch_normalization()` 逐个"手工配钥匙"把权重塞进网络。这是从 **MXNet (InsightFace 官方训练框架)** 导出再转 **PyTorch** 的典型产物——又一次暴露血统。

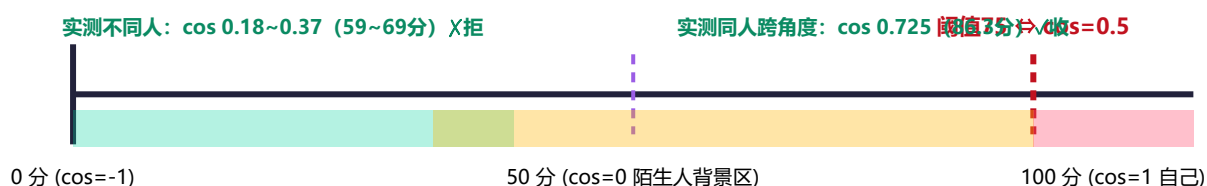
和你已有的知识连上: 人脸特征向量与文本/图像 embedding 是同一思想——[[向量与Embedding]]: 任何东西都映射成高维空间里的一个"坐标", 像不像=坐标近不近。检索靠 [[余弦相似度]], 人脸比对就是它在二维人身上的应用。区别只在: 文本用 Transformer 编码器, 人脸用 IR-ResNet。

Q7 相似度公式 $(\text{sum}(f1 \times f2) + 1) \times 50$ 的秘密

run.py 里的比对函数只有一行，却藏着三个知识点：

```
def get_similarity(feat1, feat2):  
    return (np.sum(feat1 * feat2) + 1) * 50    # 输出 0~100, 阈值 75
```

- ① $\text{np.sum}(f1 \times f2)$ 逐元素相乘再求和 = **点积**。对两个 L2 归一化的向量，**点积就等于余弦相似度** ($\cos \in [-1, 1]$)，因为分母 $\|f1\| \cdot \|f2\| = 1 \times 1$ 被归一化吃掉了；
- ② $+1) \times 50$ 是线性映射：**把 $[-1, 1]$ 的余弦拉皮成 $0 \sim 100$ 的“漂亮分数”**—— $\cos = -1$ (完全相反) $\rightarrow 0$ 分， $\cos = 0$ (无关) $\rightarrow 50$ 分， $\cos = 1$ (自己) $\rightarrow 100$ 分；
- ③ 阈值 75 倒推回去： **$75 = (\cos + 1) \times 50 \Leftrightarrow \cos = 0.5$** 。“同一人”的判定本质是“两张脸的向量夹角 $\leq 60^\circ$ ”。



实测数据 (见 Q10)：同一人跨角度 0.725，不同人 0.18~0.37，阈值 0.5 正好躺在沟里，两边留有余量。这个“同异分离度”是衡量任何人脸模型好坏的第一指标。

Q8 闭源 getPose68 里到底在做什么？（静态取证）

姿态估计 (pitch 俯仰 / yaw 偏转 / roll 滚转三个角度) 编译在闭源 DLL 里没有源码。但 `libFaceUtil.so` 没剥离符号表，用 `strings` 命令就能读到内部函数名单：

```
$ strings libFaceUtil.so | grep -v gcc  
_Z11determinantPfi      → 行列式  
_Z14solve_CHOLESKYpfiiS_iS_ → Cholesky 线性方程组求解  
_Z14JacobiSVDImpl_...   → Jacobi 奇异值分解(SVD)  
_Z9RodriguesPKfiiPf     → Rodrigues 旋转向量公式  
_Z9getPose68fffPfiRfS0_S0_ → 对外导出的姿态接口
```

这份“函数户口本”基本可以断定套路：**拿 68 个 2D 关键点去拟合一个通用 3D 人脸模板，用 SVD 最小二乘解出旋转矩阵，再用 Rodrigues 公式把矩阵转成三个欧拉角**——这是教科书级的“2D 关键点估头部姿态”经典方法 (OpenCV 的 solvePnP 同族)。实测 1.jpg 的姿态 (7.6°, 7.3°, -5.4°) (基本正脸)、2.png 的 (0.1°, -30.8°, -3.6°) (明显左转头 31°)，数值合理。

启示：闭源 ≠ 黑盒。二进制文件的符号表、字符串常量、导入导出表都能“静态取证”。当然最稳的验证还是**黑盒对拍**：给它标准输入看输出对不对 (本项目姿态输出与肉眼观察一致，通过)。

Q9 CPU 性能实测账单：50 毫秒花在哪？

用 exercise/exp_pipeline.py 对每道工序单独计时（普通桌面 CPU、无显卡、torch 2.4.1+cpu）：

工序	模型	权重体积	单脸耗时(CPU)	占比
①检测	MB-Tiny SSD slim-320	1.09 MB	≈20 ms	40%
②关键点	MobileFaceNet(64×64灰度)	3.7 MB	≈6 ms	12%
③姿态	闭源DLL(纯数学)	0.07 MB	≈1 ms	2%
④特征	IR-50	56 MB	≈21 ms	42%
⑤比对	numpy 点积	—	<0.1 ms	~0%

两个工程结论：① **检测和特征几乎平分秋色**，想提速要么减检测分辨率、要么换更小的特征网（如 MobileFaceNet 做特征）；② 对一张图里的 N 张脸，②③④按脸数线性增长而①只跑一次，多人合影时检测占比会被摊薄。另有**冷启动成本约 2~3 秒**（加载 torch+三个权重），长驻进程无感，CLI 一次性调用很扎眼——做工具时要常驻进程化。

Q10 验货实验复盘：变装认得出吗？别人分得开吗？

实验A·同脸变装：把 1.jpg 检出的人脸裁出来做 5 种“摧残”——水平翻转、旋转10°、高斯模糊、提亮1.4倍、缩小4倍再放大——再互相打分：

	原图脸	翻转	旋转	模糊	提亮	缩放	
原图脸	100.0	99.1	99.9	98.6	98.8	98.1	← 全部 ≥97.3 判同人

结论：常规变装完全打不倒它（这五招恰好都是“像素级变化、几何级不变”的操作，对齐+归一化设计就是冲它们来的）。真正的弱点要用角度刁钻的照片才能打出来——同人跨 31° 时分数掉到 86，掉得比模糊狠得多。

实验B·不同人区分：加进两张公开授权样例头像（randomuser.me 的男性/女性标准头像，不采集任何真实人脸）：

1.jpg vs 2.png	cos=0.725 → 86.3	✓同人（官方demo就是同一人两个角度！）
1.jpg vs 男头像	cos=0.374 → 68.7	✗正确拒绝
1.jpg vs 女头像	cos=0.181 → 59.0	✗正确拒绝
男头像 vs 女头像	cos=0.193 → 59.6	✗正确拒绝

意外彩蛋：官方自带的 demo 图 1.jpg 和 2.png 其实是**同一个人的两张照片**（run.py 跑出 86.27 判“same person”；姿态数据 7.3° vs -30.8° 佐证是同一人不同角度）——所以官方示例演示的从头到尾都是“同人验证”，从未演示过“拒绝陌生人”。我们在外部加“别人”才补全了这块验收。

Q11 "数据不出设备"的价值与边界

价值：云端人脸 API 的问题不是"识别不准"，而是你的脸变成了别人服务器上的一份**不可撤回的资产**——可能被缓存、被二次训练、被泄露。本地方案把生物特征数据锁在你自己的进程里：断网可用、无调用费、无账号、特征向量若不落盘则用完即逝。

但"本地跑"≠隐私自动安全，还有三件事要自己做：

风险点	如果不处理	该做什么
特征向量落盘	脸向量泄露虽拼不回照片，但可做跨系统追踪关联	加密存储或派生 ID（参考 [[派生会话ID与结构脱敏]]：sha256 现场派生，不留原值）
活体攻击	拿你照片/屏幕就能骗过摄像头	本 SDK 无活体检测（宣传有、实货无），高安全场景必须另配
闭源DLL的信任	无法审计对齐环节是否有数据外传逻辑（本SDK实测无网络代码，风险低）	抓包验证 + 用开源方案（如 insightface）替换该环节

和知识库连上：这与 [[个人记忆流水线]] 的隐私纪律同一条河——数据在本地加工、密钥不外流、对外只出"结论"不出"原料"。本地人脸识别只是把这条纪律应用在生物特征上。[[本地推理引擎]] (ollama) 是同一哲学在 LLM 上的版本：权重在自家厨房，数据不出门。

Q12 它给我什么能力？能拿它做什么？

能力清单（实测有效）：任意图片找人脸并标框 → 输出 68 点关键点 → 输出三轴头部姿态角 → 输出 256 维脸向量 → 0~100 分同人判定 → 全流程离线纯 CPU。Python 三个函数接入：`GetImageInfo(图, 最多几张脸)`、`get_similarity(向量1, 向量2)`、阈值 75 自己定。

**相册自动聚类**
批量提特征→向量聚类→按人分文件夹。离线照片不想传云盘？正好。

**考勤/签到小工具**
注册几张贴桌子的脸，摄像头抓帧比对过阈值打勾。注意需加活体对抗。

**照片自动打码**
检测框反查坐标，对脸区域高斯模糊——发视频前保护路人隐私。

**学人脸识别的教具**
五个环节源码齐全、体积小大到能逐行读完，改一处参数立刻看到行为变化。

定位对比：对比 dlib 的 face_recognition 库（要装 C++/cmake，Windows 劝退）它开箱即跑；对比 InsightFace（精度高、模型全、社区大，但坑多）它胜在极简；对比各种云 API 它赢在隐私+免费。一句话：**玩具级场景的最短路径，产品级场景的教材**——商用前必须解决 LICENSE 缺失和闭源 DLL 两个法律/审计问题，届时首选换成 insightface 系开源权重。

动手实验区（全部真机验证过，CPU 即可）

实验 1 • 跑通官方基线 (1 分钟)

怎么操作 建 Python 3.9+ 环境, `pip install -r requirements.txt`, 进 repo 目录 `python run.py`

目的 验证整条流水线在你机器上能通 (顺便验证"不需要联网")

实现了 对 test/1.jpg 和 test/2.png 各走一遍五步流水线并比对

✅ **预期结果:** 输出 `score = 86.27 → same person`, 全程约 4.6 秒 (大头是首次加载权重), 无任何网络请求。

实验 2 • 逐工序计时+可视化 (exp_pipeline.py)

怎么操作 `python exp_pipeline.py ../repo/test/1.jpg` (脚本自动切到 repo 根目录跑)

目的 看清每道工序的输入输出和耗时, 把 68 个关键点画到人脸

实现了 检测 20ms / 关键点 6ms / 姿态 1ms / 特征 22ms; 特征 256 维、L2范数=1.0; 标注图存 exercise/annotated_output.jpg

✅ **预期结果:** 控制台打出四段耗时和 68 点; 打开标注图能看到绿色人脸框 + 蓝黄两色关键点贴在五官上。

实验 3 • 同脸变装矩阵 (exp_similarity.py)

怎么操作 `python exp_similarity.py` ——裁出 1.jpg 的人脸做 5 种变换, 输出 7×7 相似度矩阵

目的 验证"同人不同照片"的鲁棒性, 理解 $score=(cos+1) \times 50$ 的取值范围

实现了 翻转/旋转/模糊/提亮/缩放五种变换互相打分 97.3~100, 全部 ≥ 75 判同人

✅ **预期结果:** 矩阵对角线恒为 100 (自己 vs 自己), 同脸变装列全部 97+。如果某行低于 75, 说明那个变换破坏了几何一致性——值得想想为什么。

实验 4 • 不同人区分度 (exp4_different_person.py)

怎么操作 下载 2 张公开授权样例头像 (randomuser.me, 允许 mockup 使用, 不采集真实人脸), `python exp4_different_person.py`

目的 补上官方 demo 缺失的"拒绝陌生人"验收, 拿到真实的同人/异人分数沟壑

实现了 同人跨角度 86.3 ✓; 三个不同人组合 59.0~68.7 X 全部正确拒绝

✅ **预期结果:** 4×4 矩阵呈现清晰的块状: 同人块 ≥ 86 , 异人块 ≤ 69 , 阈值 75 在沟里。同时确认 1.jpg/2.png 官方图实为同一人。

坑清单 (实测踩过)

- ① **相对路径炸弹:** SDK 三个模块加载权重全用 `"./face_xxx/..."` 相对路径, 从别的目录 import 直接 `FileNotFoundError` ——必须先 `os.chdir(repo根目录)`;
- ② **Python 版本:** requirements 钉死 `torch==2.4.1`, 最高支持 Python 3.12, 新版 3.14 装不上 (需 CPU 版装 3.12 venv);
- ③ **Tensor/ndarray 边界:** 检测返回 `torch.Tensor`, 画图前要 `b.numpy()`, 且特征网络开着梯度要 `detach()`;
- ④ **DLL 位数:** FaceUtil.dll 是 x64, 32 位 Python 加载即崩; Linux 用同名 .so; Mac 无支持;
- ⑤ **法律风险:** 无 LICENSE 文件 = 默认保留所有权利, "开源"只是 README 徽章说的, 商用前先发邮件问作者要授权。

